

ASSIGNMENT 9

QUESTION 1

The Iris dataset is a classic example for demonstrating classification algorithms. It consists of 150 samples of iris flowers belonging to three species: Setosa, Versicolor, and Virginica, with four input features (sepal and petal length/width). Use SVC from sklearn.svm on the Iris dataset and follow the steps below:

- Load the dataset and perform train-test split (80:20).
- Train three different SVM models using the following kernels: Linear, Polynomial (degree=3), RBF
- Evaluate each model using:
 - Accuracy
 - Precision
 - Recall
 - F1-Score
- Display the confusion matrix for each kernel.
- Identify which kernel performs the best and why

SOLUTION

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
)

iris = datasets.load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
kernels = ["linear", "poly", "rbf"]

results = {}

for kernel in kernels:
    if kernel == "poly":
        model = SVC(kernel=kernel, degree=3)
    else:
        model = SVC(kernel=kernel)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average="weighted")
    recall = recall_score(y_test, y_pred, average="weighted")
    f1 = f1_score(y_test, y_pred, average="weighted")
    cm = confusion_matrix(y_test, y_pred)
```

```

    results[kernel] = {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1_score": f1,
        "confusion_matrix": cm,
    }

for kernel, metrics in results.items():
    print(f"Kernel: {kernel}")
    print(f"Accuracy: {metrics['accuracy']:.4f}")
    print(f"Precision: {metrics['precision']:.4f}")
    print(f"Recall: {metrics['recall']:.4f}")
    print(f"F1-Score: {metrics['f1_score']:.4f}")
    print("Confusion Matrix:")
    print(metrics["confusion_matrix"])
    print("\n")

```

OUTPUT

```

Kernel: linear Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

```

Kernel: poly
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

```

Kernel: rbf
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

```

QUESTION 2

SVM models are highly sensitive to the scale of input features. When features have different ranges, the algorithm may incorrectly assign higher importance to variables with larger magnitudes, affecting the placement of the separating hyperplane. Feature scaling ensures that all attributes contribute equally to distance-based computations, which is especially crucial for kernels like RBF or polynomial.

(a) Use the Breast Cancer dataset from `sklearn.datasets.load_breast_cancer`.

- (b) Train an SVM (RBF kernel) model with and without feature scaling (StandardScaler). Compare both results using:
- Training accuracy
 - Testing accuracy
- (c) Discuss the effect of feature scaling on SVM performance.

SOLUTION

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

data = datasets.load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

svm_no_scaling = SVC(kernel="rbf", random_state=42)
svm_no_scaling.fit(X_train, y_train)
y_train_pred_no_scaling = svm_no_scaling.predict(X_train)
y_test_pred_no_scaling = svm_no_scaling.predict(X_test)
train_accuracy_no_scaling = accuracy_score(y_train, y_train_pred_no_scaling)
test_accuracy_no_scaling = accuracy_score(y_test, y_test_pred_no_scaling)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm_with_scaling = SVC(kernel="rbf", random_state=42)
svm_with_scaling.fit(X_train_scaled, y_train)
y_train_pred_with_scaling = svm_with_scaling.predict(X_train_scaled)
y_test_pred_with_scaling = svm_with_scaling.predict(X_test_scaled)
train_accuracy_with_scaling = accuracy_score(y_train, y_train_pred_with_scaling)
test_accuracy_with_scaling = accuracy_score(y_test, y_test_pred_with_scaling)

print("SVM without Feature Scaling:")
print(f"Training Accuracy: {train_accuracy_no_scaling:.4f}")
print(f"Testing Accuracy: {test_accuracy_no_scaling:.4f}")
print("\nSVM with Feature Scaling:")
print(f"Training Accuracy: {train_accuracy_with_scaling:.4f}")
print(f"Testing Accuracy: {test_accuracy_with_scaling:.4f}")
```

OUTPUT

SVM without Feature Scaling:
Training Accuracy: 0.9143
Testing Accuracy: 0.9474

SVM with Feature Scaling:
Training Accuracy: 0.9890
Testing Accuracy: 0.9825